Coding Challenge: AI Chemistry Video Request Service

Before You Start

- Completion is **not** the end goal.
- You should allocate about `90–120 minutes`, and you do not need to spend more time than that.
- You must use Claude Code, Codex, Cursor Agent, or a similar agentic coding harness while working. If you do not use one, we will not proceed with the application.
- We care about your planning, architectural decision-making, and ability to guide AI coding agents effectively.
- Do not try to maximize surface area. Be deliberate about what you focus on.
- If you feel like taking more time to polish it up, feel free to send us a version of the feature that you are proud of.
- While you work, record:
  - your full screen
  - your face

You can do this by opening a Zoom meeting by yourself and recording the session locally.

We want to see how you actually approach the problem in real time, not just the final artifact.

Scenario

Growtrics is building AI-native learning experiences.

For this challenge, imagine a learner wants to request a short educational video explaining a chemistry concept. The product should feel like a backend video request service: a client submits a concept request, the backend processes it as a video-generation job, and the client can check when the video is ready.

Latency is not a major issue. The video does not need to generate instantly. It is fine for the request to take time, as long as the backend handles that waiting state clearly and exposes it through the API.

This is a backend challenge. Do **not** build a frontend. If you need a client for demonstration, use curl, a simple script, API docs, Postman, or another lightweight API client.

Product Requirements

Build a working backend prototype of an AI chemistry video request service.

The product must have:

- a **FastAPI backend**
- an API endpoint where a client can request a chemistry concept explanation video
- an asynchronous video-generation flow
- a way to list requested videos or jobs
- a visible status for each requested video or job
- a way to retrieve or open a completed video explanation artifact
- visual content and audio for the explanation, similar to how a normal short educational video would feel
- a clear backend boundary for job state, generation logic, persistence, and artifacts

The generated explanation can be simple, but it should be coherent, useful, and visibly connected to the learner's query.

In-memory persistence is acceptable if the boundary is clean. A simple local file/artifact store is acceptable. Mocked or partly simulated generation is acceptable if the service design makes it clear where real AI/video-generation providers would be plugged in.

Your goal is not just to generate *any* video. You should aim to create the best visual explanation you can at the cheapest reasonable cost. We will treat both cost-efficiency and visual quality as success metrics.

The pipeline should also be reliable. LLMs and generative media tools are non-deterministic, so reliable output requires proper engineering around them. It should not only work for one lucky concept or fail randomly across repeated runs. Design the generation flow so the required concepts can be processed consistently, with understandable failure states, validation checks, retries, fallbacks, guardrails, or other safeguards where appropriate. The goal is not to pretend generation is deterministic, but to engineer the system so video quality remains consistently good despite that non-determinism.

Required Chemistry Queries

Your prototype must support these three learner queries end-to-end:

- `How does the pH scale work?`
- `Why do atoms form covalent bonds?`
- `What is the difference between ionic and covalent bonding?`

These three are the required scope. You do not need to support other subjects or other chemistry topics.

That said, design the backend so it is clear how other STEM topics could be added later.

What We Are Evaluating

Product judgement

We want to see whether you can translate a product requirement into a sensible backend slice.

You should decide what to build, what to fake, what to simplify, and what to leave out.

Architecture and planning

Before and during implementation, show your thinking clearly.

We are looking for evidence that you can:

- choose a practical backend architecture
- define a clean API and job lifecycle
- decide what the video artifact should contain
- reason about async job state
- reason about cost tradeoffs in the generation pipeline
- design for repeatable, reliable generation despite LLM/media-generation non-determinism
- keep the implementation small without making it incoherent
- make tradeoffs intentionally rather than randomly

Reliability under non-determinism

This is one of the most important things we look at, and it is where most submissions are weakest.

LLMs and generative media tools return something different every run. A pipeline that produces a good video once, and a broken or empty one on the next attempt, is not a working pipeline.

We are looking for evidence that you can:

- treat non-determinism as an engineering problem rather than an inconvenience
- validate generated output before it reaches the learner, rather than assuming the model got it right
- define understandable failure states instead of failing silently or half-way
- apply retries, fallbacks, guardrails, or quality gates where they earn their place
- get consistent results across repeated runs of the same concept, not just one good run

We pay attention to whether your design would hold up across repeated runs, not only to your best single output.

AI-agent workflow

Strong guidance and planning usually produce better AI-agent output.

We are looking for evidence that you can:

- give the agent clear implementation plans
- inspect and correct generated code
- break work into coherent steps
- verify behaviour instead of blindly trusting the model
- recover when the model produces weak or broken output

If the agent struggles, we are interested in how you steer it, not whether you get frustrated with it.

Quality

Testing, debugging, and reliability matter. Use your judgement on how much to implement within the time limit.

We are not asking for production polish, but the demo should be understandable and should not be a pile of disconnected pieces.

For this backend role, we will pay particular attention to API clarity, job-state handling, error handling, observability, and whether the generation boundary could realistically evolve into a production service.

We will also evaluate whether your generated videos are visually pleasing, educationally clear, and cost-conscious. A strong solution should explain what you optimized for, what each generated artifact roughly costs or would cost in production, and how the backend avoids flaky generation behaviour caused by non-deterministic LLM or media-generation outputs.

How to Submit

> Send your submission by email to `careers@growtrics.ai`.
>
> This is the main point of contact for this challenge. Please use it for your submission and for any questions along the way.
>
> When you submit, CC `praveen.k@growtrics.ai` and `wayne.le@growtrics.ai`.

Deliverables

Please send back:

- a codebase containing the FastAPI backend
- a short `README.md` with setup, run, API, and test instructions
- a short architecture note explaining the job lifecycle, persistence/artifact boundary, and AI/video-generation boundary

- a demo video or API walkthrough showcasing the three required chemistry concepts
- the three best generated videos committed into the repo, along with the input learner query that produced each video
- your GitHub link
- please make sure `careers@growtrics.ai`, `praveen.k@growtrics.ai`, and `wayne.le@growtrics.ai` have read access to the repository
- a zip file containing your work
- a Google Drive link to the recording of:
  - your full screen
  - your face

The committed generated videos are important because we want to track exactly what the system produced at the time of submission.

Again, completion is not the point. We care more about whether you were strategic, whether you chose the right slice to tackle, and how effectively you used Claude Code, Codex, Cursor Agent, or a similar agentic coding harness while coding.